CPU Utilization as a Software-Only Thermal Proxy for Multi-Tenant Edge AI Inference: A 13-Hour Characterization and Coupling Law on Raspberry Pi 5

Manu Nicholas Jacob

***Abstract*—Thermal-aware control of edge AI inference usually presumes instrumented power sensing, which most deployed single-board computers (SBCs) lack. We ask a narrower, practical question: on a current-generation SBC running real inference, how well does *CPU utilization alone*, a fully software-observable signal, predict SoC temperature, and is the relationship stable enough to control against? From a 13-hour, four-block campaign on Raspberry Pi 5 (232,136 samples at 5 Hz: characterization, load sweeps, eight two-tenant configurations, and a six-hour stability run) we obtain a cross-validated *CPU–thermal coupling law*, $T \approx 45.6\,^{\circ}\mathrm{C} + 0.175\,^{\circ}\mathrm{C}/\% \cdot U$ ($R^2 = 0.93$), with per-block slopes within $\pm 8\%$, leave-one-block-out error of 0.84–$2.45\,^{\circ}\mathrm{C}$, a response that settles within one 200 ms control tick, and a slope that transfers across a six-month recalibration within 5%. Compared like-for-like against the standard structures (lumped-RC, autoregressive, and multi-signal models), the law matches every sensor-free, actuatable alternative (all collapse to 1.6–1.8 °C), while sensor-feedback forecasters are $2.7\times$ tighter at one step but cannot answer allocation what-ifs and presuppose the sensor. The law converts a temperature budget into a utilization budget with no power instrumentation. We release the runtime and full dataset.**

***Index Terms*—Edge computing, edge AI, thermal management, Raspberry Pi, empirical characterization, reproducible research, ONNX Runtime.**

I. Introduction

EDGE AI deployments increasingly run continuous neural-network inference on small single-board computers (SBCs) in always-on perception pipelines. These platforms operate under a junction-temperature ceiling enforced by modest cooling, a fixed pool of cores shared among concurrent workloads, and application latency requirements. The thermal constraint is distinctive because it integrates load over time: sustained inference heats the SoC toward a firmware throttle threshold beyond which frequency is cut and latency degrades nonlinearly.

Thermal-aware runtime control requires a model that predicts temperature from observable signals. The natural predictor is board power, and a large body of work assumes an instrumented power rail; most deployed SBCs ship none and cannot be retrofitted in the field. The practical question for a deployable controller is thus "*what can I predict temperature from using only software-observable telemetry*."

This paper answers that question empirically for Raspberry Pi 5, the current-generation reference SBC. We show that CPU utilization, available from the operating system with no special hardware, predicts SoC temperature with $R^2 = 0.93$ through a simple linear law that is stable across workloads and time, and whose dynamics are fast enough for real-time use. Concretely, we make four contributions:

- **A cross-validated coupling law** for Raspberry Pi 5, $T \approx 45.6 + 0.175\,U$, with per-block slope consistency establishing it as a property of the platform, not one workload (§IV).
- **Its dynamics**: the response settles within a 200 ms control tick, so the steady-state law is directly usable in real time (§V).
- **A like-for-like comparison** against lumped-RC, AR/ARX, and multi-signal models under one protocol (§VI).
- **A multi-tenant characterization** over eight two-tenant pairs, and an open runtime plus 232,136-sample dataset (§VII, §IX).

We are explicit about scope. We report no board-power measurements: the measurement platform had no power sensor attached, and we present the coupling law as the software-only alternative that this absence motivates. We also note that the CPU frequency governor was pinned to `performance` (constant 2.4 GHz), so the law characterizes the fixed-frequency regime; we return to both points in §X.

II. Related Work

DNN inference on edge SBCs. Efficient architectures [1], optimized runtimes [2], and edge benchmarks [5] characterize inference latency on SBCs, but none provide a reusable utilization-to-temperature model for control.

Thermal- and power-aware computing. Dynamic thermal management [3], [4] and DVFS operate below the application layer and assume privileged frequency control or instrumented power, neither available to an application-level controller on a stock SBC.

M. N. Jacob received the dual B.S. degree in Electrical Engineering and Computer Engineering from the University of Massachusetts Amherst, Amherst, MA, USA, in 2025 (e-mail: manunicholasjacob@gmail.com; ORCID: 0009-0007-6589-6572).

Runtime, configurations, raw telemetry, and analysis scripts are released at https://github.com/manunicholasjacob/pi5-thermal-proxy.

Software thermal and power estimation. The established structures are: performance-counter power models feeding a lumped RC thermal model, as in Isci and Martonosi's runtime power monitoring [6] and Lee and Skadron's counter-driven temperature sensing through HotSpot [7]; autoregressive forecasters fit to the temperature sensor itself, as in Coskun et al.'s ARMA-based proactive management [8]; and calibrated utilization-based power models on battery devices such as PowerTutor [9]. These assume either a calibrated per-platform counter-to-power model, or continuous access to the very sensor a proxy is meant to replace. What is missing for deployed SBCs is a quantitative, cross-validated utilization-to-temperature law measured on current hardware under real inference, published with its dataset. Section VI evaluates representatives of each structure above on our data under one protocol.

Positioning. We contribute (i) a named, cross-validated coupling law on real hardware, (ii) its dynamic characterization, (iii) a like-for-like accuracy comparison against the standard prediction structures (§VI), and (iv) a multi-tenant thermal dataset, all released. We intentionally scope out power (not collected in this campaign) and DVFS (governor pinned), and we are explicit that these are the boundaries of the claims.

III. Measurement Methodology

A. Platform

Raspberry Pi 5 (Broadcom BCM2712, quad-core Arm Cortex-A76 @ 2.4 GHz), 64-bit Raspberry Pi OS, ONNX Runtime CPU execution provider. SoC temperature is read from `vcgencmd`/`sysfs`; CPU and memory utilization from the OS. The CPU governor is pinned to `performance`, so core frequency is a constant 2.4 GHz throughout (we verified a single frequency value across all samples); the campaign therefore characterizes the fixed-frequency operating regime typical of a latency-oriented inference deployment.

B. Runtime and telemetry

Inference is served by a modular Python runtime that hosts ONNX Runtime workers and a telemetry thread sampling temperature, per-core CPU utilization, and memory at 5 Hz, writing time-aligned rows to Apache Parquet. Each experiment block runs sequentially; a fresh session logs each block.

C. Experiment design

The campaign comprises four blocks over 13 continuous hours: **Block 1**, single-workload characterization (MobileNetV2 and ResNet-18 at two resolutions, fixed rate); **Block 2**, a MobileNetV2 load sweep populating the utilization–temperature space; **Block 3**, eight ordered pairs of two concurrent ONNX workloads (SqueezeNet, MobileNetV2, ResNet-18, EfficientNet-lite0, YOLOv5n) on four cores; **Block 5**, a six-hour continuous MobileNetV2 run testing for drift.

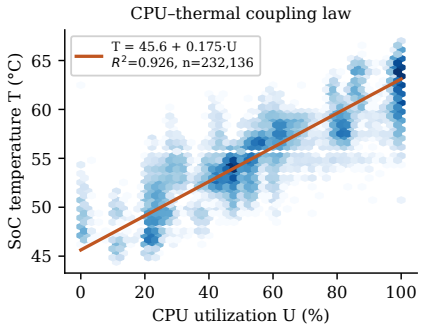

Fig. 1. CPU–thermal coupling on Raspberry Pi 5 (232,136 samples, log-density hexbins). SoC temperature is linear in CPU utilization with $R^2 = 0.93$; the fitted law is Eq. (1).

D. Dataset provenance and de-duplication

The raw capture contains nine Parquet files. Each experiment block was logged by two concurrent collector processes started ∼90 s apart; both recorded the same physical run at the same 4.95 Hz effective rate. We therefore de-duplicate to one collector per run, yielding **232,136 unique time-aligned samples** (the naive sum over all nine files, 471,827, double-counts). All results below use the de-duplicated set. This provenance detail is stated so that the released dataset is interpreted correctly.

E. What we measure and what we do not

We report SoC temperature, CPU utilization, and memory. We *do not* report board power: no sensor was read during the campaign, and rather than publish an unvalidated surrogate we omit power entirely (§X). Per-inference latency was not logged, so multi-tenant results (§VII) are thermal, utilization, and memory characterizations.

IV. The CPU–Thermal Coupling Law

A. Global fit

Across all 232,136 samples, SoC temperature is a strong linear function of CPU utilization (Fig. 1):

$$T \;=\; 45.63\,^{\circ}\mathrm{C} \;+\; 0.1748\,\tfrac{^{\circ}\mathrm{C}}{\%}\cdot U, \qquad (1)$$

with Pearson $r = 0.962$, $R^2 = 0.926$, and a residual standard deviation of $1.33\,^{\circ}$C. The slope is estimated extremely tightly: its 95% confidence interval is $[0.1746, 0.1750]\,^{\circ}$C/%, a consequence of the large sample size and the genuine linearity of the relationship. Equation (1) says that each additional percent of aggregate CPU utilization raises steady-state SoC temperature by about $0.175\,^{\circ}$C, from an idle intercept near $45.6\,^{\circ}$C.

B. Cross-block validation

A law useful for control must generalize beyond the workload that produced it. We refit Eq. (1) in a leave-one-block-out protocol: train on three blocks, predict the held-out block. Prediction RMSE ranges from $0.84\,^{\circ}$C (stability block) to $2.45\,^{\circ}$C (single-workload block), and the per-block slopes cluster tightly around the global value (Fig. 2): 0.184 (Block 3),

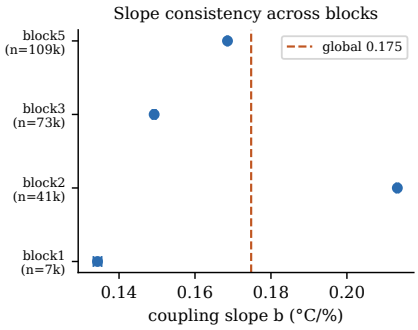

Fig. 2. Per-block coupling slope with 95% confidence intervals against the global slope (dashed). The slope is consistent across four independent experiment blocks, establishing platform-level (not workload-specific) coupling.

0.177 (Block 1), 0.160 (Block 2), 0.180 (Block 5) °C/%. The intercept is similarly stable (45.1–46.6 °C). That the slope varies by less than ±8% across blocks driven by different models, resolutions, request rates, and tenancy establishes the coupling as a property of the *platform* rather than of any one workload, which is the prerequisite for using it as a fixed control model.

C. Portability across time and ambient

A six-month out-of-sample check: recalibrating the law on the same unit in September 2026 (different season, ambient, and background state; a 150 s three-tenant soak against a 45 s idle baseline) yields a slope of 0.183 °C/% against the campaign's 0.175, within 5%, while the intercept moved by only 0.1 °C. The slope, which is what a controller inverts, transfers; the intercept is ambient- and cooling-dependent and should be re-measured at deployment, a 45-second idle read.

D. Interpretation for control

Equation (1) is directly actionable. A controller that knows a thermal budget $T_{\max}$ can invert the law to a *utilization budget*, $U_{\max} = (T_{\max} - 45.63)/0.1748$, with U the system-wide utilization on the operating system's 0–100 scale. Full load projects to a steady state of 63.1 °C, so a cap at the 75 °C firmware-throttle level cannot bind at this ambient with stock cooling; the campaign's observed maximum was 67.0 °C (§VIII). Operator guard-band caps below 63 °C do bind, and the inversion turns each into a concrete budget: $U_{\max} \approx 54\%$ for a 55 °C cap. Computing it requires no power sensor: utilization is read from the OS.

V. Thermal Dynamics

Equation (1) is a steady-state relationship; a real-time controller also needs to know how quickly temperature tracks a change in utilization. We estimate a first-order thermal response, $\dot{T} = (T_{ss}(U) - T)/\tau$, from the six-hour stability run by regressing the per-sample temperature increment on the instantaneous gap between the law's steady-state target and the current temperature. The resulting time constant is $\tau \lesssim 1$ s, at or below our 200 ms (5 Hz) sampling interval, so the exact value is resolution-limited. The practical consequence: on this platform the die temperature tracks utilization changes essentially within one control tick, so a controller can treat the steady-state law (1) as instantaneously valid for allocation decisions. Two refinements keep this honest. First, a slower component exists: a ∼10 s exponentially-weighted utilization improves leave-one-block-out RMSE by 0.12 °C (§VI), the signature of heat-spreader mass superposed on the fast die response, and a sustained load moves the operating point over minutes even though the per-tick response is immediate. Second, the fitted first-order time constant on this data is $\tau \approx 1$–2 s (§VI), consistent with the resolution-limited estimate above.

TABLE I
Leave-one-block-out RMSE (°C, mean over held-out blocks). "What-if" = can predict the effect of a proposed allocation change.

Model	Signals	What-if	1-step	Sensor-free
Coupling law (1)	U	yes	1.76	**1.76**
+ EWMA (∼10 s)	U	yes	–	**1.64**
Multi-signal static	U, mem	yes	–	1.65
Lumped RC (τ=1–2 s)	U (+T)	yes	0.67	1.71
ARX(1)	T, U	rollout	0.66	1.71
AR(1)	T	no	**0.65**	–

VI. Comparison with Standard Prediction Structures

The natural question for any proxy is how it compares with the established alternatives. We implement the standard structures from §II on the released dataset and evaluate all of them under the identical leave-one-block-out protocol: a HotSpot-class first-order lumped RC model driven by utilization [7] (time constant by grid search, steady-state gain by least squares); a Coskun-class autoregressive forecaster AR(1) on the temperature sensor [8] and its ARX extension with utilization as the exogenous input; a multi-signal static regression on utilization plus memory occupancy (the counter-model structure of [6] restricted to signals a stock SBC exports); and an exponentially-filtered variant of our own law. Table I reports both tasks that matter: 200 ms-ahead forecasting *with* the temperature sensor in the loop, and sensor-free prediction from software signals alone (the input a sensorless allocation controller actually consumes), obtained from the dynamic models by rolling them out on utilization without temperature feedback.

With the sensor available, short-horizon forecasters are 2.7× tighter (0.65 vs. 1.76 °C at one step), but they answer the wrong question for allocation: an AR-family model has no term through which a proposed duty-cycle change enters, and it requires the sensor whose absence motivates a proxy. Every sensor-free, actuatable structure instead lands in the same 1.6–1.8 °C band. Rolling out the dynamic RC or ARX models on utilization without temperature feedback recovers no accuracy over the static law, because at $\tau \approx 1$–2 s the plant settles within a control tick and the dynamics carry almost no information at 5 Hz; the richer static models buy 0.1 °C. On this class of platform the single-signal law is the accuracy frontier for sensor-free thermal prediction, at two constants,

TABLE II
MULTI-TENANT SUMMARY (BLOCK 3, EIGHT TWO-TENANT PAIRS, 73,508 SAMPLES). COUPLING-LAW PREDICTION USES EQ. (1) AT THE OBSERVED MEAN UTILIZATION.

Quantity	Value
Concurrent workload pairs	8
Mean CPU utilization	78.6 %
Mean SoC temperature (observed)	59.3 °C
Mean SoC temperature (Eq. 1 prediction)	59.4 °C
Mean resident memory	1,162 MB
Thermal-budget violations (>75 °C)	0

and the frontier is adequate for the job: a 1.6–1.8 °C error band is small against the 8 °C minimum margin observed in §VIII and the multi-degree guard-bands operators set.

VII. MULTI-TENANT CHARACTERIZATION

Block 3 runs eight ordered pairs of two concurrent ONNX workloads on the four cores, spanning SqueezeNet, MobileNetV2, ResNet-18, EfficientNet-lite0, and YOLOv5n (73,508 samples; Table II). The coupling law predicts the multi-tenant mean temperature within 0.1 °C from the observed mean utilization ($45.63 + 0.1748 \times 78.6 = 59.4$ vs. 59.3 °C observed), evidence that the single law also governs the multi-tenant regime. No pair exceeded the thermal budget; the binding constraint under two tenants is memory and core count, not temperature.

VIII. CONSTRAINT HEADROOM

Across the 13-hour campaign, SoC temperature ranged 44.4–67.0 °C with **zero** excursions above a 75 °C budget, and the six-hour stability block showed no monotonic drift. For the workloads tested with stock cooling, thermal control on Raspberry Pi 5 is therefore *preventive* rather than reactive; and because Eq. (1) predicts the utilization at which a budget binds, that margin is maintainable from software-observable utilization alone.

IX. REPRODUCIBILITY ARTIFACT

We release the runtime, block configurations, the full de-duplicated 232,136-sample dataset, and scripts that regenerate every number, figure, and the comparison in Table I. The schema and de-duplication procedure (§III) are documented so the data is interpreted correctly.

X. LIMITATIONS

No power measurement. The campaign includes no board-power data. We note for precision that Raspberry Pi 5 does expose PMIC rail telemetry in software (`vcgencmd pmic_read_adc`); this campaign predates our use of that interface, and validating the law against PMIC power is future work. The released dataset originally carried placeholder power fields written by the collector; they contained no measurement and have been removed from the artifact, with a note documenting the removal.

Fixed frequency. The governor was pinned to `performance`, so Eq. (1) characterizes the constant-2.4-GHz regime. Under a dynamic governor, frequency becomes a second thermal driver and the single-variable law would need a frequency term.

Single board and ambient. Results are for one Raspberry Pi 5 unit under laboratory ambient with stock cooling. The intercept (idle temperature) is ambient-dependent; the slope is expected to be more portable but is not verified across units or ambients here.

No per-inference latency. The telemetry does not include per-request latency, so multi-tenant interference is characterized thermally and in memory, not in latency or throughput.

XI. CONCLUSION

On Raspberry Pi 5, CPU utilization predicts SoC temperature through a tight, cross-validated linear law, $T \approx 45.6 + 0.175\,U$ ($R^2 = 0.93$), stable across four experiment blocks, across six months and ambients (slope within 5%), with dynamics that settle within one control tick. Compared like-for-like against the standard prediction structures, the law matches the sensor-free accuracy of lumped-RC and ARX models rolled out on utilization (the dynamics add nothing at this time constant), while sensor-feedback forecasters, though 2.7× tighter at one step, cannot answer allocation what-ifs and presuppose the sensor. The law lets an application-level controller reason about thermal headroom, inverting a temperature budget to a utilization budget, using only software-observable telemetry, with no power sensor of the kind most deployed SBCs lack. We release the runtime and the full 232,136-sample dataset to support reproducible, sensor-free thermal-control research on commodity edge hardware.

ACKNOWLEDGMENT

No external funding was received for this work. All experiments, data, analyses, and conclusions are the author's own. Generative AI (Claude, Anthropic) assisted with manuscript editing, LaTeX, and some analysis; all of its output was reviewed and verified by the author.

REFERENCES

[1] M. Sandler *et al.*, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proc. IEEE/CVF CVPR*, 2018, pp. 4510–4520.
[2] ONNX Runtime developers, "ONNX Runtime," 2024. [Online]. Available: https://onnxruntime.ai
[3] D. Brooks and M. Martonosi, "Dynamic Thermal Management for High-Performance Microprocessors," in *Proc. HPCA*, 2001, pp. 171–182.
[4] K. Skadron *et al.*, "Temperature-Aware Microarchitecture," in *Proc. ISCA*, 2003, pp. 2–13.
[5] C. Banbury *et al.*, "MLPerf Tiny Benchmark," in *Proc. NeurIPS Datasets Benchmarks*, 2021.
[6] C. Isci and M. Martonosi, "Runtime Power Monitoring in High-End Processors: Methodology and Empirical Data," in *Proc. IEEE/ACM MICRO-36*, 2003, pp. 93–104.
[7] K.-J. Lee and K. Skadron, "Using Performance Counters for Runtime Temperature Sensing in High-Performance Processors," in *Proc. IEEE IPDPS (Workshop on High-Performance, Power-Aware Computing)*, 2005.
[8] A. K. Coskun, T. S. Rosing, and K. C. Gross, "Proactive Temperature Management in MPSoCs," in *Proc. ACM/IEEE ISLPED*, 2008, pp. 165–170.
[9] L. Zhang *et al.*, "Accurate Online Power Estimation and Automatic Battery Behavior Based Power Model Generation for Smartphones," in *Proc. IEEE/ACM/IFIP CODES+ISSS*, 2010, pp. 105–114.